Tabla de contenidos

1. Módulo `os`
2. *Paths* y nombres de archivos
3. Problema de portabilidad de *paths*
4. Navegación entre directorios
5. Un ejemplo de lectura y escritura básico
6. Pathlib

Módulo `os`

El módulo `os` de Python nos provee una interfaz portable para ejecutar operaciones relacionadas al sistema operativo y al sistema de archivos. En lo inmediato lo utilizaremos para ejecutar operaciones sobre las rutas (*path*) de los archivos, utilizando `os.path`. El manejo apropiado de los *paths* usando este módulo nos ayudará a hacer nuestro código más portable, permitiéndonos ejecutar código sin problemas en distintos sistemas operativos.

```
import os
```

Paths y nombres de archivos

Un *path* (o ruta) nos indica el lugar donde se encuentra un archivo o un directorio dentro de un árbol que representa nuestro sistema de archivos. Como notarás, las carpetas y archivos en un computador se almacenan jerárquicamente: carpetas (o directorios) dentro de otras carpetas, y archivos dentro de carpetas. Un *path* es la dirección de un directorio o archivo dentro de esta jerarquía. La siguiente figura muestra de forma gráfica un ejemplo de esta jerarquía.

Entonces, un ejemplo de ruta es: `'Pedro/Libros/calculo.pdf'`. Esta especifica la dirección de un archivo `calculo.pdf`, que está dentro del directorio `Libros`, que a su vez está dentro de la carpeta `Pedro`.

Es necesario trabajar con *paths* cuando un programa quiere acceder a un archivo para leerlo o para crear un nuevo archivo y guardarlo. A través de los *paths* es que un programa es capaz de especificar a cuál archivo o directorio quiere acceder dentro de un computador.

Un *path* puede ser **absoluto** o **relativo**.

- Un ***path* absoluto** es la dirección de un archivo o directorio desde la **raíz** del sistema de archivos. Puede pensarse como las instrucciones para llegar desde el comienzo del computador. Un *path* absoluto **siempre** comienza con la dirección del directorio principal o directorio raíz (*root*). Este, en muchos sistemas operativos se representa con el carácter `/` , pero en otros puede variar (en Windows puede ser `\` o `C:/`). Un *path* absoluto tiene el mismo significado de manera independiente del directorio en donde se está ejecutando el programa. Tiene la ventaja que la ruta no presenta ambigüedades, pero requiere que la ruta exista **exactamente igual** en todos los sistemas de archivos en que se ejecuta el programa.

- Un ***path* relativo nunca** comienza con el carácter de directorio raíz e indica una dirección **relativa a cierto directorio**. Un *path* relativo se interpreta a partir de algún directorio específico, que corresponde por lo general al que contiene al programa actual. Tiene la ventaja que permite referenciar a un directorio de manera más simple, ya que no recurre a toda la jerarquía del sistema de archivos. Pero aún así requiere cuidado al momento de ejecutar desde un directorio distinto al esperado.

Por ejemplo, si seguimos el ejemplo de la estructura de la imagen superior, podemos definir rutas para el archivo `python.pdf`:

- `"/Users/Pedro/Libros/python.pdf"` es la ruta absoluta del archivo, ya que detalla desde el directorio raíz la dirección del archivo. Esta ruta es única, no hay otra ruta absoluta para el archivo.
- `"Users/Pedro/Libros/python.pdf"` es la ruta relativa del archivo, **relativa desde el directorio raíz `"/"`**.
- `"Libros/python.pdf"` es la ruta relativa del archivo, **relativa desde el directorio `"/Users/Pedro"`**.
- `"python.pdf"` es la ruta relativa del archivo, **relativa desde el directorio `"/Users/Pedro/Libros"`**.

Es importante notar las sutilezas de ambos tipos de *paths* al crear un programa. Recordemos que al ejecutar un programa escrito en Python, esta **ejecución se realiza en cierto directorio** (o carpeta) del computador. Al usar rutas absolutas, no importa desde qué directorio se ejecute el programa, ya que la dirección del *path* no cambia. A diferencia de las rutas relativas, que sí cambian dependiendo de la carpeta donde se ejecute. Por otro lado, un programa no necesariamente se ejecuta siempre en el mismo equipo (a veces se ejecuta en nuestro computador, o a veces en el computador del ayudante). Es por eso que **usar rutas absolutas dificulta el funcionamiento de un programa, ya que pueden estar fijas a un computador específico**, con una estructura única que no se replica en otros computadores. Por todo lo anterior, no se aconseja escribir programas que usen rutas absolutas para acceder a archivos, sino que **deben utilizar solo rutas relativas**.

```
# Path absoluto

path_absoluto = '/home/archivo.txt'

with open(path_absoluto, 'rt') as archivo:
    lineas = archivo.readlines()

lineas
```

En el código anterior, se intenta abrir un archivo llamado `archivo.txt` una carpeta llamada `home`, ubicada en el directorio raíz (`/`) del sistema de archivos del computador donde se está ejecutando el código. Este código lanzará una excepción, a menos que esta carpeta y este archivo existan.

```
## Path relativo

path_relativo = 'data/archivo.txt'
```

```
with open(path_relativo, 'rt') as archivo:
    lineas = archivo.readlines()

lineas
```

En este código, se intenta abrir un archivo llamado `archivo.txt` que está dentro de una carpeta llamada `data`, que está ubicada en la carpeta donde se está ejecutando este código Python. Este archivo debería leerse sin problemas, ya que el repositorio donde se encuentra este material incluye a esta carpeta y a este archivo.

Un *path* es la dirección de un archivo o de un directorio. Este siempre se divide en dos partes:

- El nombre del directorio o `dirname`, que es la carpeta donde se encuentra el archivo o directorio objetivo.
- El nombre de archivo o directorio objetivo, *filename* o `basename`, que es el nombre del archivo, incluyendo su extensión, o directorio.

En el siguiente código, el módulo `os.path` permite separar un *path* en `dirname` y `basename`.

```
path1 = '/carpeta1/carpeta2/imagen.jpg'

dirname1 = os.path.dirname(path1)
basename1 = os.path.basename(path1)

print(f'path: {path1}')
print(f'dirname: {dirname1}')
print(f'basename: {basename1}')

path2 = 'f1/f2/f3'

dirname2 = os.path.dirname(path2)
basename2 = os.path.basename(path2)

print(f'path: {path2}')
print(f'dirname: {dirname2}')
print(f'basename: {basename2}')
```

Extensiones de archivo

Los nombres de archivo suelen terminar con una secuencia, típicamente de tres caracteres, que aparece después de un punto, por ejemplo `.txt`, `.jpg`, `.pdf`, `.mp3` y `.avi`. Esta secuencia de letras se conoce como **extensión** del archivo y sirve para dos objetivos:

1. Darle una *pista* al usuario sobre el tipo de archivo de que se trata, para saber qué hacer con él. Por ejemplo, cómo abrirlo.
2. Darle una *pista* al sistema operativo para saber con qué programa leer el archivo.

```
# Podemos usar la función `splitext` para separar la extension del
resto del nombre de archivo
```

```
nombre_sin_extension, extension = os.path.splitext(basename1)
print(nombre_sin_extension)
print(extension)
```

Hay que tener en cuenta que, si bien la extensión del archivo sirve para darnos información acerca del tipo de archivo, ésta es **parte del nombre de archivo** y es **sólo una convención**. Una extensión informa del tipo de archivo, pero **no determina el tipo del archivo**.

Por ejemplo, a continuación escribiremos un archivo de texto y lo guardaremos con extensión `.jpg`, que indica que el archivo es una imagen en formato JPEG. Para un usuario puede verse extraño que un archivo de texto tenga una extensión de imagen, sin embargo, el archivo seguirá siendo un archivo de texto.

```
path = 'data/archivo_de_texto.jpg'

with open(path, 'w') as f:
    f.writelines(['linea 1\n', 'linea 2\n', 'linea 3\n'])
```

Si intentas abrir el archivo generado, puede que tu sistema operativo intente erróneamente abrirlo con un visor de imágenes. Sin embargo, si lo abres con tu editor de texto favorito, deberías poder leerlo adecuadamente, ya que es un archivo de texto.

Algunos sistemas operativos vienen configurados por defecto para ocultar la extensión de los archivos. Se recomienda fuertemente cambiar esta configuración para poder ver el nombre de archivo completo y evitar confusiones al leer y escribir archivos.

Problema de portabilidad de *paths*

Como se ha mencionado anteriormente, hay diferencias en los caracteres de separación entre sistemas operativos, por ejemplo la ruta `"Users/Pedro/Libros/python.pdf"` abriría correctamente un archivo en un sistema Unix (Linux y macOS), pero puede fallar en Windows, ya que la ruta equivalente para ese sistema operativo sería `"Users\Pedro\Libros\python.pdf"`.

Esto trae un problema de portabilidad de programas, que si explicitan rutas utilizando alguna de estas escrituras y luego se intentan ejecutar en otro sistema operativo, puede que no funcione, ya que la ruta es inválida en ese ambiente.

El módulo `os.path` provee muchas funcionalidades para reescribir rutas usando los caracteres de separación nativos del sistema operativo dónde se ejecute el programa. Por ejemplo, el método `os.path.join`:

```
ruta = os.path.join("Users", "Pedro", "Libros", "python.pdf")
ruta
```

El método genera automáticamente la ruta equivalente al concatenar los argumentos que se le den, utilizando el separador del sistema operativo dónde se ejecute. Se recomienda **fuertemente** trabajar con *paths* de esta forma, y así asegurar portabilidad de sus programas.

Navegación entre directorios

La librería `os` también nos provee funciones para ver el contenido de directorios y recorrer sus distintos archivos y directorios.

La más conocida, es la función `listdir` que al entregarle una ruta de un directorio, entrega una lista con los nombres de directorios y archivos que se encuentran directamente dentro de ese directorio. Por ejemplo, al ejecutarlo con la ruta `"data"`, vemos el contenido de la carpeta `data` dentro de este repositorio:

```
lista_de_contenidos = os.listdir("data")
lista_de_contenidos
```

Vemos los nombres de tres archivos, y el de una carpeta `gato` que se utiliza en un ejemplo siguiente. Notar que no aparecen en esta lista los contenidos de `gato`, ya que `listdir` solo lista los contenidos directamente almacenados.

```
lista_de_contenidos = os.listdir(os.path.join("data", "gato"))
lista_de_contenidos
```

En cambio, la función `walk` nos permite poder obtener las rutas de un directorio, de sus subdirectorios y de sus archivos. Esto nos permite poder navegar dentro de una carpeta y ver, recursivamente, todo lo que contiene.

El siguiente código muestra como se utiliza:

```
for raiz, directorios, archivos in os.walk("data", topdown=True):
    print("Raíz:", raiz)
    print()
    print("Archivos:")
    for archivo in archivos:
        print(os.path.join(raiz, archivo))
    print()
    print("Directorios:")
    for directorio in directorios:
        print(os.path.join(raiz, directorio))
    print("-" * 30)
```

El parámetro `topdown` nos permite decidir si la navegación sobre una carpeta será desde el directorio raíz (`topdown=True`) o desde sus elementos hojas (`topdown=False`).

Un ejemplo de lectura y escritura básico

Finalmente, se muestra un ejemplo de lectura y escritura de archivos de texto simple. El siguiente código busca leer y guardar tableros del juego **Gato**.

Comenzaremos leyendo un tablero existente. Dentro de la carpeta `gato` (dentro de la carpeta `data`) existe un archivo de texto simple con un tablero de Gato: `juego_1.txt`. Para abrirlo,

usamos la función `open` de Python, que recibe la ruta de un archivo. Utilizaremos la función `os.path.join` para definir la ruta de forma portable:

```
ruta_juego_1 = os.path.join("data", "gato", "juego_1.txt")
archivo = open(ruta_juego_1, "rt")
archivo.readlines()
```

El argumento `"rt"` especifica el modo de apertura del archivo. La `"r"` significa que es en modo de lectura (*read*), y la `"t"` en forma de texto. Más adelante en el curso estudiaremos otra forma de lectura y escritura de archivos.

Mediante el método `readlines` obtenemos una lista de las líneas del archivo de texto. Vemos que el tablero se representa mediante filas en cada línea, y cada una se separa por comas para cada posición del tablero. Hay una `"X"` o una `"O"` para representar jugadas de los jugadores, o `"-"` para un espacio vacío.

Antes de procesar este contenido, debemos cerrar el archivo que acabamos de abrir:

```
archivo.close()
```

Cada vez que se use la función `open` de esta forma, el archivo usado debe ser cerrado utilizando `close`. Una alternativa al uso de `close`, es abrir un archivo utilizando un ambiente generado con la sentencia `with`, como a continuación:

```
with open(ruta_juego_1, "rt") as archivo:
    lineas = archivo.readlines()
```

La ventaja de esta forma, es que al salir de la indentación generada por la sentencia `with` el archivo se cierra automáticamente y no es necesario utilizar `close`. Si vemos la variable generada, notamos que el contenido del archivo se obtuvo de la misma forma y se almacenó:

```
lineas
```

Notamos que esta forma de almacenar el tablero sigue siendo poco conveniente, ya que usa *strings* para cada fila. El siguiente código limpia y separa cada fila en una lista con las posiciones separadas:

```
tablero = []
for linea in lineas:
    fila = linea.strip().split(',')
    print(fila)
    tablero.append(fila)
```

El método `strip` por defecto remueve espacios o saltos de líneas al comienzo o al final de un *string*, en este caso remueve `"\n"` de cada línea. Luego `split` separa el string según un carácter, en este caso, utilizamos `","` para separar.

Así, generamos una lista de dos dimensiones, con las posiciones del tablero ordenadas:

```
tablero
```

 ProTip: Además de usar `.strip()` para eliminar los saltos de línea al final de cada línea, también es posible evitar este paso si se abre el archivo con el parámetro `newline=''`.

```python with open(ruta_juego_1, "rt", newline='') as archivo: lineas = archivo.readlines()

for linea in lineas: fila = linea.split(',') # ya no es necesario usar .strip()
```

Ahora aplicaremos una nueva jugada por el jugador `"O"`, modificando una de las posiciones del tablero:

```
tablero[2][2] = 'O'

for fila in tablero:
    print(fila)
```

Y finalmente guardaremos el resultado en otro archivo. Nuevamente, definiremos la ruta utilizando `os.path.join`, y para escribir en archivos también podemos usar la sentencia `with`:

```
ruta_juego_2 = os.path.join("data", "gato", "juego_2.txt")

with open(ruta_juego_2, "wt") as archivo:
    for fila in tablero:
        fila_en_texto = ",".join(fila) + "\n"
        print(fila_en_texto)
        archivo.write(fila_en_texto)
```

Aquí, el método `join` de un *string* recibe una lista de *strings* y genera otro *string* más largo que es el resultado de concatenar los *strings* pero alternando el contenido usando el *string* original, en este caso, `","`.

El resultado del tablero quedó almacenado en `"data/gato/juego_2.txt"`.

Pathlib

Este es un módulo de Python que ofrece clases capaces de representar las rutas del sistema de archivos. Es decir, utiliza una interfaz orientada a objetos para manipular rutas de archivos y directorios de manera eficiente. Podrás encontrar la documentación completa en español en el siguiente link

```
from pathlib import Path
```

En el código anterior, estamos importando la clase Path con todos sus atributos y métodos.

```
# Creamos la ruta instanciando un objeto de la clase Path
path = Path("data/archivo.txt")
```

Al ser un objeto, cuenta con diversos atributos y métodos. A continuación detallaremos los más relevantes.

```
print("Nombre del archivo:", path.name)

print("Extensión del archivo:", path.suffix)

print('Es un directorio:', path.is_dir())

print('Es un archivo:',path.is_file())

path_absoluto = path.absolute()
print('Ruta absoluta:', path_absoluto)

if path.exists():
    print("El archivo existe en la ruta indicada")

print('\nLeer el contenido de un archivo de texto:')
contenido = path.read_text()
print(contenido)

print('\nRecorriendo el directorio de la carpeta data:')
directorio = Path("./data/")
for archivo in directorio.iterdir():
    print(archivo)
```

El objeto instanciado de la clase Path también permite escritura y lectura, usado en conjunto con la función `os` de Python.

```
with path.open(mode='r') as archivo:
    contenido = archivo.read()
    print(contenido)

path1 = Path("./data/nuevo_archivo.txt")
with path1.open(mode='w') as archivo:
    archivo.write("Podría funcionar mejor?\n")
    archivo.write("Recordar que el modo 'w' sobre escribe lo
existente!")

with path1.open(mode='r') as archivo:
    contenido = archivo.read()
    print(contenido)
```